

NEXUS — Industry-Level Software Implementation Plan

Engineering the Campus Super-App: Architecture, Infrastructure & Roadmap

TABLE OF CONTENTS

1. System Architecture Overview
 2. Technology Stack
 3. Database Architecture
 4. Microservices Design
 5. Mobile Application Architecture
 6. Authentication & Authorization
 7. Payment System Implementation
 8. Real-Time Infrastructure
 9. Trust & Safety Engineering
 10. Geolocation & Rides Infrastructure
 11. Search & Recommendations
 12. Notification System
 13. Analytics & Data Pipeline
 14. DevOps, CI/CD & Infrastructure
 15. Security Architecture
 16. Development Phases & Sprints
 17. Quality Assurance Strategy
 18. Scaling Strategy
 19. Disaster Recovery
 20. Compliance Engineering
 21. Admin & Operations Tools
 22. Third-Party Integrations
 23. Post-Launch Maintenance
-

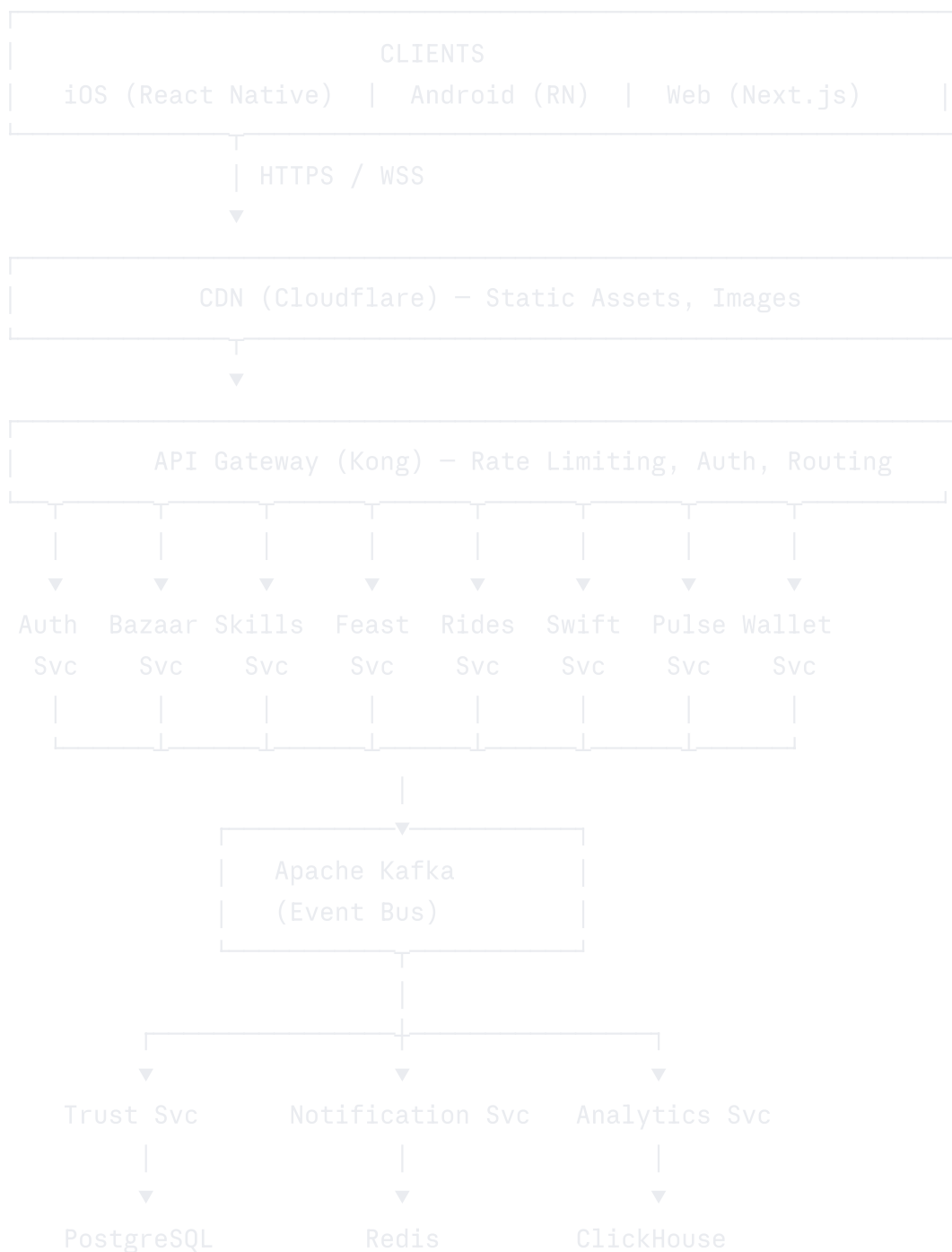
1. SYSTEM ARCHITECTURE OVERVIEW

1.1 Architecture Philosophy

NEXUS is built on five core engineering principles:

1. **Domain-Driven Design (DDD):** Each business domain (Bazaar, Skills, Feast, Rides, Pulse) is an independent bounded context with its own data, logic, and APIs.
2. **Event-Driven Architecture:** Services communicate asynchronously through Kafka events wherever possible — loose coupling, high resilience.
3. **API-First:** Every feature is a versioned API, enabling web, mobile, and future third-party access simultaneously.
4. **Security by Design:** Auth, authorization, encryption, and audit logging are foundational constraints — never afterthoughts.
5. **Progressive Scalability:** Works on ₹20,000/month infra in Year 1; horizontally scales to 100M users without architectural rewrite.

1.2 High-Level System Diagram



(Primary DB)	(Cache/PubSub)	(OLAP/BI)
MongoDB	Elasticsearch	S3 (Data Lake)
(Documents)	(Search)	

1.3 Deployment Evolution

Phase	Infra	Monthly Cost
Year 1 (MVP, 1 campus)	AWS ECS Fargate + RDS + ElastiCache	₹15,000-25,000
Year 2 (5 campuses)	ECS + Aurora + Redis Cluster	₹80,000-1,50,000
Year 3+ (20+ campuses)	Kubernetes (EKS) + Aurora Global + Multi-region	₹5L-15L

2. TECHNOLOGY STACK

2.1 Core Stack Decisions

Layer	Technology	Rationale
Mobile	React Native (Expo SDK 50+)	Single iOS+Android codebase; huge talent pool; fast iteration
Web	Next.js 14 (React)	SSR/SSG for SEO; PWA-capable; same language as mobile
Backend (Primary)	Node.js + TypeScript + Fastify	Async I/O ideal for real-time; full JS stack; typed safety
Backend (Performance)	Go (Golang)	Rides matching + Wallet service — microsecond latency matters
API Gateway	Kong OSS	Mature, plugin-rich, K8s-native
Primary Database	PostgreSQL 15	ACID for financial transactions; battle-tested
Cache + PubSub	Redis 7	Sessions, rate limiting, real-time pub/sub, queues
Document Store	MongoDB	Flexible schema for listings, messages, event structures
Search Engine	Elasticsearch 8	Full-text + geo search; listing discovery
Message Broker	Apache Kafka	Event streaming; audit log; analytics pipeline
Object Storage	AWS S3 + CloudFront	Images, documents, static assets at CDN speed

Layer	Technology	Rationale
Container Runtime	Docker + Kubernetes	Consistent environments; auto-scaling
CI/CD	GitHub Actions + ArgoCD	Automated testing + GitOps deployment
Monitoring	Grafana + Prometheus + Loki	Metrics, alerts, log aggregation
Error Tracking	Sentry	Real-time error tracking across all services

2.2 Mobile App Tech Stack

```
React Native (Expo)
├─ State: Zustand (global) + React Query (server state)
├─ Navigation: Expo Router (file-based)
├─ UI: Custom Design System + React Native Paper
├─ Maps: react-native-maps (Google Maps SDK)
├─ Real-Time: Socket.io client
├─ Payments: Razorpay RN SDK
├─ Storage: MMKV (encrypted local storage)
├─ Push Notifs: Expo Notifications (FCM + APNs)
├─ Camera: expo-camera + expo-image-picker
├─ Analytics: Mixpanel RN SDK
├─ Biometrics: expo-local-authentication
└─ OTA Updates: Expo Updates (instant patches without App Store review)
```

2.3 Backend Service Stack per Module

Service	Runtime	DB
Auth Service	Node.js/TS (Fastify)	PostgreSQL + Redis
Bazaar Service	Node.js/TS (Fastify)	PostgreSQL + MongoDB
Skills Service	Node.js/TS (Fastify)	PostgreSQL
Feast Service	Node.js/TS (Fastify)	PostgreSQL + Redis
Rides Service	Go (Gin)	PostgreSQL + Redis
Swift Service	Node.js/TS (Fastify)	PostgreSQL + Redis
Wallet Service	Go (Gin)	PostgreSQL (strict ACID)
Pulse Service	Node.js/TS (Fastify)	MongoDB + PostgreSQL

Service	Runtime	DB
Search Service	Node.js/TS (Fastify)	Elasticsearch
Trust Service	Node.js/TS (Fastify)	PostgreSQL
Notification Service	Node.js/TS + BullMQ	Redis + PostgreSQL
Analytics Service	Python (FastAPI)	ClickHouse + S3

3. DATABASE ARCHITECTURE

3.1 Core Schema Design

```
sql
```

-- USERS & PROFILES

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email VARCHAR(255) UNIQUE NOT NULL, -- university email  
  phone VARCHAR(15) UNIQUE NOT NULL,  
  phone_verified BOOLEAN DEFAULT FALSE,  
  email_verified BOOLEAN DEFAULT FALSE,  
  status VARCHAR(20) DEFAULT 'active', -- active, suspended, banned  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

```
CREATE TABLE student_profiles (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  full_name VARCHAR(100) NOT NULL,  
  srm_student_id VARCHAR(20) UNIQUE NOT NULL,  
  department VARCHAR(100),  
  year_of_study SMALLINT,  
  hostel_block VARCHAR(20),  
  room_number VARCHAR(10),  
  avatar_url TEXT,  
  trust_score DECIMAL(3,2) DEFAULT 3.00,  
  verification_level SMALLINT DEFAULT 1,  
  -- 1=email, 2=student_id, 3=face_match, 4=aadhaar  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- LISTINGS (BAZAAR)

```
CREATE TABLE listings (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  seller_id UUID REFERENCES users(id),  
  title VARCHAR(200) NOT NULL,  
  description TEXT,  
  category VARCHAR(50) NOT NULL,  
  condition VARCHAR(20), -- new, like_new, good, fair, rough  
  price DECIMAL(10,2) NOT NULL,  
  listing_type VARCHAR(20) DEFAULT 'fixed', -- fixed, negotiable, auction, rental  
  status VARCHAR(20) DEFAULT 'active', -- active, sold, reserved, expired  
  images JSONB DEFAULT '[]', -- array of S3 URLs  
  ai_suggested_price DECIMAL(10,2),  
  campus_id UUID NOT NULL,  
  is_promoted BOOLEAN DEFAULT FALSE,  
  promoted_until TIMESTAMPTZ,  
  view_count INT DEFAULT 0,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  expires_at TIMESTAMPTZ DEFAULT (NOW() + INTERVAL '30 days')
```

```

);

-- TRANSACTIONS (Financial source of truth)
CREATE TABLE transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  buyer_id UUID REFERENCES users(id),
  seller_id UUID REFERENCES users(id),
  reference_id UUID,          -- listing_id, task_id, ride_id, order_id
  module VARCHAR(20) NOT NULL, -- bazaar, skills, feast, swift, rides
  amount DECIMAL(10,2) NOT NULL,
  platform_fee DECIMAL(10,2) NOT NULL,
  seller_amount DECIMAL(10,2) NOT NULL,
  status VARCHAR(30) DEFAULT 'initiated',
  -- Flow: initiated → payment_held → in_progress → completed | disputed | refunded
  gateway_order_id VARCHAR(100),
  gateway_payment_id VARCHAR(100),
  escrow_release_at TIMESTAMPTZ,
  dispute_window_ends TIMESTAMPTZ,
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
) PARTITION BY RANGE (created_at); -- Monthly partitions

-- WALLET (Financial backbone)
CREATE TABLE wallets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) UNIQUE,
  balance DECIMAL(12,2) DEFAULT 0.00 CHECK (balance >= 0),
  coin_balance INT DEFAULT 0,
  locked_amount DECIMAL(12,2) DEFAULT 0.00, -- in escrow
  daily_limit DECIMAL(10,2) DEFAULT 2000.00,
  monthly_limit DECIMAL(10,2) DEFAULT 10000.00,
  kyc_level SMALLINT DEFAULT 0,
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE wallet_ledger (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  wallet_id UUID REFERENCES wallets(id),
  transaction_id UUID REFERENCES transactions(id),
  entry_type VARCHAR(20), -- credit, debit, lock, unlock, fee
  amount DECIMAL(10,2) NOT NULL,
  balance_after DECIMAL(12,2) NOT NULL,
  description TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- APPEND-ONLY: No UPDATE or DELETE ever permitted on wallet_ledger

```

```

-- DISPUTES
CREATE TABLE disputes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_id UUID REFERENCES transactions(id),
  raised_by UUID REFERENCES users(id),
  reason VARCHAR(50),
  description TEXT,
  evidence JSONB DEFAULT '[]', -- S3 URLs + text
  status VARCHAR(30) DEFAULT 'open',
  resolution VARCHAR(30), -- refund_buyer, release_seller, partial_refund
  resolved_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  resolved_at TIMESTAMPTZ
);

-- RATINGS
CREATE TABLE ratings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_id UUID REFERENCES transactions(id),
  rater_id UUID REFERENCES users(id),
  ratee_id UUID REFERENCES users(id),
  score SMALLINT CHECK (score BETWEEN 1 AND 5),
  review_text TEXT,
  is_flagged BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- KEY INDEXES
CREATE INDEX idx_listings_campus ON listings(campus_id, category) WHERE status = 'active';
CREATE INDEX idx_transactions_buyer ON transactions(buyer_id, created_at);
CREATE INDEX idx_transactions_seller ON transactions(seller_id, created_at);
CREATE INDEX idx_wallet_ledger_wallet ON wallet_ledger(wallet_id, created_at);

```

3.2 Redis Architecture

```

Redis Keyspace Design:
├─ DB 0 – Sessions
|   session:{userId}:{deviceId} → JWT refresh token (TTL: 30d)
├─ DB 1 – Rate Limiting
|   ratelimit:{endpoint}:{userId} → count (TTL: 60s)
├─ DB 2 – OTPs
|   otp:{phone} → bcrypt(OTP) (TTL: 5min)
|   otp:{email} → bcrypt(OTP) (TTL: 10min)
├─ DB 3 – Application Cache

```

```
| listing:{listingId} → JSON (TTL: 5min)
| campus:{campusId}:listings:{category}:p{page} → IDs (TTL: 2min)
| user:trust:{userId} → trust score (TTL: 1hr)
└─ DB 4 – Real-Time Pub/Sub
| Channels: ride:{rideId}, task:{taskId}, order:{orderId}, campus:{campusId}
└─ DB 5 – Job Queues (BullMQ)
| Queues: notifications, emails, sms, escrow-releases (delayed jobs)
└─ DB 6 – Leaderboards
    trust_scores:campus:{campusId} → Sorted Set (userId → score)
```

3.3 Sharding Strategy (Year 3+)

Multi-Tenant Approach: Schema-per-Campus

```
└─ nexus_global          → users, wallets, trust_scores (cross-campus entities)
└─ nexus_srm_ktr         → all campus-specific data (listings, orders, rides)
└─ nexus_vit_vellore    → isolated campus data
└─ nexus_amrita_cb       → isolated campus data
```

Campus schemas are fully isolated. Global schemas handle cross-campus portability. Aurora PostgreSQL Global Database handles replication across regions.

4. MICROSERVICES DESIGN

4.1 Service Communication Patterns

Synchronous (gRPC): Service-to-service calls requiring immediate response

- Bazaar → Wallet: lock escrow before confirming order
- Rides → Trust: validate driver score before matching

Asynchronous (Kafka): Events for non-critical flows

Kafka Topics:

nexus.users.*	User lifecycle events
nexus.transactions.*	All financial events (immutable audit)
nexus.listings.*	Marketplace events
nexus.rides.*	Ride lifecycle
nexus.tasks.*	Swift task lifecycle
nexus.notifications.*	Notification triggers
nexus.analytics.*	Analytics pipeline
nexus.moderation.*	Content moderation queue
nexus.trust.*	Trust score update triggers

4.2 Wallet Service — Critical Design

```
go

// Go - Wallet escrow lock (atomic PostgreSQL transaction)
func (s *WalletService) LockEscrow(ctx context.Context, req *LockEscrowRequest) error {
    return s.db.WithTransaction(ctx, func(tx *sql.Tx) error {
        // SELECT FOR UPDATE prevents concurrent modifications
        var wallet Wallet
        err := tx.QueryRowContext(ctx,
            `SELECT id, balance, locked_amount FROM wallets
            WHERE user_id = $1 FOR UPDATE`, req.BuyerID).Scan(
            &wallet.ID, &wallet.Balance, &wallet.LockedAmount)
        if err != nil { return err }

        if wallet.Balance < req.Amount {
            return ErrInsufficientBalance
        }

        // Atomic: debit balance, credit locked_amount
        _, err = tx.ExecContext(ctx,
            `UPDATE wallets SET
            balance = balance - $1,
            locked_amount = locked_amount + $1,
            updated_at = NOW()
            WHERE id = $2`, req.Amount, wallet.ID)
        if err != nil { return err }

        // Append-only ledger entry
        _, err = tx.ExecContext(ctx,
            `INSERT INTO wallet_ledger
            (wallet_id, transaction_id, entry_type, amount, balance_after, description)
            VALUES ($1, $2, 'lock', $3, $4, $5)`,
            wallet.ID, req.TransactionID, req.Amount,
            wallet.Balance - req.Amount,
            fmt.Sprintf("Escrow lock for transaction %s", req.TransactionID))
        return err
    })
}
```

4.3 Rides Service — Matching Algorithm (Go)

```
go
```

```
// Real-time ride matching with PostGIS
func (s *RidesService) MatchRide(ctx context.Context, request *RideRequest) (*MatchR
// Find available drivers within expanding radius
radii := []float64{500, 1000, 2000, 5000} // meters

for _, radius := range radii {
    drivers, err := s.db.QueryContext(ctx, `
        SELECT user_id,
               ST_Distance(location::geography, ST_MakePoint($1,$2)::geography)
               trust_score, is_women_only
        FROM active_drivers
        WHERE ST_DWithin(location::geography, ST_MakePoint($1,$2)::geography, $3
              AND campus_id = $4
              AND is_available = true
              AND ($5 = false OR is_women_only = true)
        ORDER BY (dist * 0.6 + (5.0 - trust_score) * 0.4 * 1000) ASC
        LIMIT 5`,
        request.PickupLng, request.PickupLat, radius,
        request.CampusID, request.WomenOnly)

    if err != nil || len(drivers) == 0 { continue }

    // Notify top 3 drivers via WebSocket, await first acceptance
    return s.notifyAndAwaitAcceptance(ctx, drivers[:min(3,len(drivers))], request)
}
return nil, ErrNoDriversAvailable
}
```

4.4 Trust Score Engine

typescript

```
// Trust Score: composite of 5 weighted factors
interface TrustFactors {
  avgRating: number; // 0-5, from all received ratings
  completionRate: number; // 0-1, tasks/transactions completed vs accepted
  verificationLevel: number; // 1-4, higher = more verified
  accountAgeDays: number; // normalized to 0-1 over 365 days
  disputeFreeScore: number; // 0-1, penalized for each lost dispute
}

function computeTrustScore(factors: TrustFactors): number {
  const score =
    (factors.avgRating / 5) * 0.40 +
    factors.completionRate * 0.25 +
    (factors.verificationLevel / 4) * 0.15 +
    Math.min(factors.accountAgeDays / 365, 1) * 0.10 +
    factors.disputeFreeScore * 0.10;

  return parseFloat((score * 5).toFixed(2)); // Normalize to 0-5
}

// Recomputed nightly via cron; Redis sorted set updated post-compute
```

5. MOBILE APPLICATION ARCHITECTURE

5.1 Project Structure (Feature-Sliced Design)

```
nexus-mobile/
├── app/                                # Expo Router (file-based routing)
│   ├── (auth)/                        # Login, Register, Verify screens
│   ├── (tabs)/                        # Main bottom tab navigator
│   │   ├── bazaar/                   # Buy/Sell module
│   │   ├── feast/                   # Food delivery module
│   │   ├── rides/                   # NEXUS Rides
│   │   ├── swift/                   # Errands/tasks
│   │   ├── skills/                  # Services marketplace
│   │   └── pulse/                   # Events + communities
│   ├── wallet/                       # Wallet & payments
│   ├── profile/                      # User profile + settings
│   └── chat/                         # In-app messaging
├── features/                          # Business logic per domain
│   ├── bazaar/
│   │   ├── api.ts                  # API calls
│   │   ├── hooks.ts               # React Query hooks
│   │   ├── store.ts               # Zustand state
│   │   └── types.ts               # TypeScript types
```

```

|   ├── wallet/
|   ├── rides/
|   └── ...
└── shared/
    ├── api/           # Axios client, interceptors, error handling
    ├── ui/            # Design system: Button, Card, Input, etc.
    ├── hooks/         # useAuth, useLocation, useWebSocket
    ├── utils/         # formatCurrency, formatDate, etc.
    └── assets/        # Fonts, icons, images

```

5.2 State Management Pattern

typescript

```

// Zustand: Global auth state (synchronous, persisted)
const useAuthStore = create<AuthStore>()(
  persist(
    (set, get) => ({
      user: null,
      tokens: null,
      isAuthenticated: false,
      login: async (credentials) => {
        const { user, tokens } = await authApi.login(credentials);
        set({ user, tokens, isAuthenticated: true });
      },
      logout: () => {
        authApi.logout(get().tokens?.refresh);
        set({ user: null, tokens: null, isAuthenticated: false });
      },
    }),
    { storage: createJSONStorage(() => MMKV) } // Encrypted storage
  )
);

// React Query: Server state (async, cached, synchronized)
const useListings = (campusId: string, category?: string) =>
  useInfiniteQuery({
    queryKey: ['listings', campusId, category],
    queryFn: ({ pageParam }) => bazaarApi.getListings({ campusId, category, cursor:
      getNextPageParam: (last) => last.meta.cursor,
      staleTime: 2 * 60 * 1000, // 2 minutes before refetch
    });

```

5.3 Performance Targets

Metric	Target
App cold start	< 2.5 seconds
Screen transition	< 200ms
Listing page load	< 500ms (cached) / < 1.5s (fresh)
Image first paint	Blurhash placeholder within 100ms
API p99 latency	< 1 second
Crash-free rate	> 99.5%

6. AUTHENTICATION & AUTHORIZATION

6.1 Registration & Verification Flow

```
STEP 1 – Email Registration
  POST /auth/register { email: "ra1234@srmist.edu.in", phone, password }
  → Validate: email domain in allowlist (srmist.edu.in, *.srm.edu.in)
  → Check: email + phone not already registered
  → Send: Email OTP (10 min TTL) + SMS OTP (5 min TTL)

STEP 2 – Dual OTP Verification
  POST /auth/verify { email_otp, phone_otp }
  → verification_level = 1 (email verified)
  → JWT pair issued: access (15 min TTL) + refresh (30 day TTL)

STEP 3 – Student ID Verification (unlocks full features)
  POST /verify/student-id { image: base64 }
  → AWS Textract extracts: name, student ID, department
  → AI validates ID card authenticity (watermarks, format check)
  → Auto-approve if confidence > 95%; human review if 70-95%; reject if < 70%
  → verification_level = 2

STEP 4 – Face Match (optional, unlocks ₹10K wallet limit)
  POST /verify/face { selfie: base64 }
  → AWS Rekognition compares selfie to student ID card photo
  → verification_level = 3

STEP 5 – Aadhaar (optional, unlocks ₹25K wallet limit)
  → DigiLocker API integration
```

```
→ NEXUS receives only: verified=true/false (no Aadhaar number stored)
→ verification_level = 4
```

6.2 JWT Structure

```
json
{
  "sub": "uuid-here",
  "email": "ra1234@srmist.edu.in",
  "roles": ["student"],
  "campus_id": "srm-ktr",
  "verification_level": 2,
  "trust_tier": "trusted",
  "iat": 17000000000,
  "exp": 1700000900
}
```

6.3 RBAC Permission Model

```
student      → listings:crud_own, transactions:create, wallet:own
vendor       → student + feast:manage_store, feast:view_orders
driver       → student + rides:accept, rides:update_location
moderator    → student + listings:flag_any, disputes:resolve_tier2
campus_admin → moderator + users:suspend, analytics:campus
super_admin  → all permissions
```

7. PAYMENT SYSTEM IMPLEMENTATION

7.1 Payment Architecture

```
User (UPI/Card)
  |
  ▼ POST /wallet/load
Razorpay Order Created
  |
  ▼ User completes payment on Razorpay Checkout
Razorpay Webhook → POST /webhooks/razorpay
  |
  ▼ Signature verified (HMAC-SHA256)
  | Idempotency check (webhook already processed?)
  ▼
NEXUS Nodal Account (Bank)
  |
```

```
    ▼ wallet.load event → Kafka
Wallet Service credits user balance
|
▼ Confirmation push notification to user
```

7.2 Razorpay Integration

typescript

```
// Webhook handler – idempotent, signature-verified
async function handleRazorpayWebhook(req: Request, res: Response) {
  // 1. Verify signature
  const signature = req.headers['x-razorpay-signature'] as string;
  const expected = crypto
    .createHmac('sha256', process.env.RAZORPAY_WEBHOOK_SECRET!)
    .update(JSON.stringify(req.body))
    .digest('hex');

  if (signature !== expected) return res.status(401).json({ error: 'Invalid signature' });

  const paymentId = req.body.payload.payment.entity.id;

  // 2. Idempotency – prevent double processing
  const processed = await redis.get(`webhook:${paymentId}`);
  if (processed) return res.json({ status: 'already_processed' });

  // 3. Process
  if (req.body.event === 'payment.captured') {
    const { user_id, type } = req.body.payload.payment.entity.notes;
    const amount = req.body.payload.payment.entity.amount / 100; // paise → rupees

    await walletService.creditWallet({ userId: user_id, amount, reference: paymentId });
    await redis.setex(`webhook:${paymentId}`, 86400, '1'); // Mark processed, 24hr T
  }

  res.json({ status: 'processed' });
}
```

7.3 Escrow Auto-Release (BullMQ Scheduled Job)

typescript

```
// Scheduled 24 hours after payment_held status
escrowReleaseQueue.process('release', async (job) => {
  const { transactionId } = job.data;
  const tx = await db.transactions.findById(transactionId);

  if (tx.status !== 'payment_held') return; // Already resolved
  if (await db.disputes.existsForTransaction(transactionId)) return; // Dispute active

  await walletService.releaseEscrow(transactionId);
  await kafka.publish('nexus.transactions.completed', { transactionId });
});
```

7.4 GST & TDS Compliance

typescript

```
// TDS deduction – Section 194C: 1% for service providers earning > ₹30,000/year
async function processServicePayout(userId: string, grossAmount: number) {
  const annualEarnings = await db.getAnnualEarnings(userId, currentFinancialYear());

  let tdsAmount = 0;
  if (annualEarnings + grossAmount > 30000) {
    tdsAmount = grossAmount * 0.01; // 1% TDS
    await taxLedger.recordTDS({ userId, grossAmount, tdsAmount, fy: currentFY() });
  }

  const netAmount = grossAmount - tdsAmount;
  await walletService.credit(userId, netAmount);
  return { netAmount, tdsDeducted: tdsAmount };
}
```

8. REAL-TIME INFRASTRUCTURE

8.1 WebSocket Architecture

typescript

```
// Socket.io with Redis Adapter (scales across multiple server instances)
import { Server } from 'socket.io';
import { createAdapter } from '@socket.io/redis-adapter';

io.adapter(createAdapter(pubRedisClient, subRedisClient));

// Room Strategy:
// user:{userId}           → personal notifications
// transaction:{txId}      → escrow/payment status
// ride:{rideId}           → live GPS + ETA updates
// task:{taskId}           → Swift task status
// chat:{roomId}           → messaging
// campus:{campusId}       → broadcast announcements
// canteen:{canteenId}     → order queue updates for vendor tablet

// Auth middleware – validate JWT on WebSocket connection
io.use(async (socket, next) => {
  const token = socket.handshake.auth.token;
  try {
    const payload = verifyJWT(token);
    socket.data.userId = payload.sub;
    socket.data.campusId = payload.campus_id;
    socket.join(`user:${payload.sub}`);
    socket.join(`campus:${payload.campus_id}`);
    next();
  } catch {
    next(new Error('Authentication failed'));
  }
});
```

8.2 Live Ride Tracking

typescript

```
// Driver sends GPS every 3 seconds
socket.on('driver:location', async ({ rideId, lat, lng }) => {
  // Cache in Redis (TTL 30s – if no update, driver gone offline)
  await redis.setex(`driver:loc:${rideId}`, 30, JSON.stringify({ lat, lng, ts: Date.now() }));

  // Compute ETA using Google Maps Distance Matrix API (cached 30s)
  const eta = await ridesService.computeETA(lat, lng, rideDestination);

  // Broadcast to all passengers in the ride room
  io.to(`ride:${rideId}`).emit('location:update', { lat, lng, eta });
});
```

9. TRUST & SAFETY ENGINEERING

9.1 Content Moderation Pipeline

EVERY new listing image:

1. Upload → S3 (private bucket, NEXUS-signed URLs only)
2. Lambda trigger → AWS Rekognition
 - Detect: nudity, drugs, weapons, violence
 - Custom labels: counterfeit currency, exam papers
3. AWS Textract → extract text from image
 - Check for: phone numbers (to bypass escrow), prohibited terms
4. Score < threshold: AUTO-APPROVE, listing goes live
5. Score > threshold: HOLD listing, queue for human review
6. Confirmed violation: Remove + warn user
7. 3rd violation: Account suspension

EVERY service description text:

1. Profanity filter (custom SRM-context wordlist)
2. PII detection (phone numbers, UPI IDs in text = escrow bypass attempt)
3. Prohibited services: essay mills, exam paper access, plagiarism-as-service
4. Hate speech classifier (multilingual: English + Tamil + Hindi)

9.2 Fraud Detection

python

```
# Fraud risk scoring model (scikit-learn, served via FastAPI)
FEATURES = [
    'user_account_age_days',
    'verification_level',
    'trust_score',
    'transaction_amount',
    'price_deviation_pct',          # (listed_price - avg_market_price) / avg
    'seller_listing_age_hours',     # new listing + high value = risky
    'buyer_prior_disputes',
    'seller_prior_disputes',
    'is_first_txn_between_pair',
    'device_fingerprint_age_days', # fresh device on old account = flag
    'time_of_day_risk',            # 2-5 AM transactions flagged higher
]

def score_transaction(features: dict) -> dict:
    prob = model.predict_proba([extract(features)])[0][1]
    score = int(prob * 100)

    action = (
        'allow'                if score < 20 else
        'allow_with_monitoring' if score < 50 else
        'require_selfie_verification' if score < 75 else
        'block_pending_review'
    )
    return {'score': score, 'action': action}
```

9.3 Dispute Resolution Workflow

TIER 1 – Automated (< 2 hours)

Triggered by: delivery GPS not at buyer location, seller never marked shipped

Resolution: Auto-refund buyer

Cases: ~40% of disputes

TIER 2 – Campus Mediators (< 24 hours)

Trained student volunteers (senior year, vetted)

Tools: Evidence viewer (photos, chat logs, GPS data)

Decision: Refund buyer / Release to seller / Partial split

TIER 3 – NEXUS Support (< 72 hours)

NEXUS operations team

For: Complex cases, large amounts, repeat offenders

Can issue platform compensation from reserve fund

TIER 4 – Legal Escalation

For: Fraud, criminal activity, harassment
NEXUS provides full documentation to campus security / police
Zero tolerance – immediate account ban

10. GEOLOCATION & RIDES INFRASTRUCTURE

10.1 SRM Custom Campus Map

External map apps (Google Maps) have poor building-level accuracy inside SRM campus.
NEXUS builds a custom campus map:

Data Sources:

1. OpenStreetMap base data for SRM KTR area
2. Manual POI addition:
 - All academic blocks (A-G, MED, DEN, etc.) with precise coordinates
 - All hostels (Himalaya, Cauvery, Ganga, etc.) with entry points
 - All canteens with exact locations
 - Gates (Main Gate, Back Gate, Medical Gate)
 - Library, Admin Block, Sports Complex, Parking Lots
3. Host on custom tile server (MapTiler Self-Hosted)
4. Campus map updated each semester for construction changes

Integration:

- All delivery routing uses campus POIs, not arbitrary GPS
- Users select "Himalaya Hostel Block A" not raw coordinates
- Safe exchange zones pinned on campus map (CCTV-monitored)

10.2 PostGIS Spatial Queries

sql

```

-- Available rides within 1km of student's location
SELECT r.id, r.pickup_label, r.dropoff_label, r.scheduled_at,
       ST_Distance(r.pickup_location::geography,
                   ST_MakePoint($lon, $lat)::geography) AS distance_m
FROM ride_requests r
WHERE r.status = 'open'
      AND r.campus_id = $campus_id
      AND ST_DWithin(r.pickup_location::geography,
                     ST_MakePoint($lon, $lat)::geography, 1000)
ORDER BY distance_m, r.scheduled_at
LIMIT 20;

-- Spatial index (required for performance)
CREATE INDEX idx_ride_pickup_gist ON ride_requests USING GIST(pickup_location);

```

11. SEARCH & RECOMMENDATIONS

11.1 Elasticsearch Index Design

```

json
{
  "mappings": {
    "properties": {
      "title": {
        "type": "text",
        "analyzer": "english",
        "fields": {
          "suggest": { "type": "completion" }
        }
      },
      "description": { "type": "text", "analyzer": "english" },
      "category": { "type": "keyword" },
      "price": { "type": "scaled_float", "scaling_factor": 100 },
      "condition": { "type": "keyword" },
      "campus_id": { "type": "keyword" },
      "seller_trust_score": { "type": "float" },
      "is_promoted": { "type": "boolean" },
      "created_at": { "type": "date" }
    }
  }
}

```

11.2 Ranking Function

11.3 Contextual Recommendations

Campus Context Engine:

- └ Exam Week Detection (from SRM academic calendar API)
 - Boost: tutoring services, stationery, reference books
 - Suppress: event listings
- └ Time of Day
 - 07:00-09:00 → boost breakfast options in Feast
 - 20:00-23:00 → boost late snack vendors
- └ Semester Start (July, December)
 - Boost: textbook listings, stationery bundles
 - Email campaign: "Semester starter deals"
- └ Placement Season (Oct-Dec, Feb-Apr)
 - Boost: formal wear rentals, resume writing services, mock interview gigs
- └ Campus Events
 - When a fest is announced: boost food, rides, team formation

12. NOTIFICATION SYSTEM

12.1 Multi-Channel Priority Matrix

Priority	Channels	Delay	Examples
CRITICAL	Push + SMS + In-app	Immediate	SOS triggered, payment failed, account suspended
HIGH	Push + In-app	Immediate	Order ready, ride matched, task accepted, escrow released

Priority	Channels	Delay	Examples
NORMAL	Push + In-app	Batched (5 min)	New listing in watchlist, rating received, event reminder
LOW	In-app + Email	Daily digest	Coins earned, tips, weekly summary

12.2 Notification Service

typescript

```
// BullMQ producer – used by all services
async function notify(params: NotifyParams): Promise<void> {
  await notificationQueue.add('send', {
    userId: params.userId,
    title: params.title,
    body: params.body,
    data: params.data,
    channels: params.channels,
    priority: params.priority
  }, {
    priority: PRIORITY_MAP[params.priority],
    delay: params.priority === 'normal' ? 5 * 60 * 1000 : 0
  });
}

// BullMQ consumer – actually sends the notification
notificationQueue.process('send', async (job) => {
  const { userId, channels, ...payload } = job.data;
  const prefs = await db.getNotificationPrefs(userId);
  const tokens = await db.getPushTokens(userId);

  await Promise.allSettled([
    channels.includes('push') && prefs.push_enabled && sendExpoNotification(tokens,
    channels.includes('sms') && prefs.sms_enabled && sendSMS(userId, payload),
    channels.includes('email') && prefs.email_enabled && sendEmail(userId, payload),
    sendInAppNotification(userId, payload), // always
  ]);
});
```

13. ANALYTICS & DATA PIPELINE

13.1 Data Architecture

Services → Kafka → (two consumers):

1. Flink (stream) → Real-time metrics → InfluxDB → Grafana
2. S3 (batch) → Daily ETL → ClickHouse → Metabase dashboards

Mobile/Web → Mixpanel → Product analytics (funnels, retention, A/B tests)

13.2 Key Metrics Dashboard

Real-Time (Grafana):

- GMV per hour per campus
- Live active users (WebSocket connections)
- Transaction success rate (target: >99%)
- Payment gateway latency
- Fraud flags per hour

Business (Metabase — daily):

- DAU / MAU per module
- D1, D7, D30 retention cohorts
- Revenue by module and campus
- Dispute rate (target: <2%)
- Average Trust Score trend

Product (Mixpanel):

- Funnel: Register → Verify → First Transaction
- Feature adoption rates
- Screen drop-off analysis
- A/B test results

14. DEVOPS, CI/CD & INFRASTRUCTURE

14.1 Monorepo Structure

```
nexus/  
└─ apps/
```

```

|   |─ mobile/           # React Native app
|   |─ web/              # Next.js web app
|   |─ admin/            # Admin dashboard (Next.js)
|─ services/
|   |─ auth/
|   |─ bazaar/
|   |─ wallet/           # Go
|   |─ rides/            # Go
|   |─ feast/
|   |─ swift/
|   |─ skills/
|   |─ pulse/
|   |─ trust/
|   |─ notifications/
|   |─ search/
|   |─ analytics/        # Python
|─ packages/
|   |─ database/         # Shared migrations, seeds
|   |─ kafka/            # Shared Kafka client + topic definitions
|   |─ types/            # Shared TypeScript types
|   |─ utils/            # Shared utility functions
|─ infrastructure/
|   |─ terraform/        # All AWS resources as code
|   |─ kubernetes/       # K8s manifests (Year 2+)
|   |─ docker-compose.yml # Local development
|─ .github/
|   |─ workflows/        # CI/CD pipelines

```

14.2 CI/CD Pipeline

yaml

```
# GitHub Actions – runs on every PR and push to main
name: NEXUS CI/CD

on:
  push:
    branches: [main, staging]
  pull_request:
    branches: [main]

jobs:
  quality:
    steps:
      - run: npm run lint
      - run: npm run type-check
      - run: npm test --coverage
      - name: SonarCloud (code quality gate)
        uses: SonarSource/sonarcloud-github-action@master
      - name: Snyk (dependency vulnerability scan)
        uses: snyk/actions/node@master

  build:
    needs: quality
    steps:
      - run: docker build -t nexus/${{ matrix.service }}:${{ github.sha }}
      - run: docker push $ECR_REGISTRY/nexus/${{ matrix.service }}:${{ github.sha }}

  deploy-staging:
    needs: build
    if: github.ref == 'refs/heads/staging'
    steps:
      - run: argocd app sync nexus-staging --revision ${{ github.sha }}
      - run: npm run test:smoke -- --env staging

  deploy-production:
    needs: deploy-staging
    if: github.ref == 'refs/heads/main'
    environment: production # Requires manual approval in GitHub
    steps:
      - run: ./scripts/blue-green-deploy.sh ${{ github.sha }}
      - run: ./scripts/health-check.sh || ./scripts/rollback.sh
```

14.3 Local Development Setup

```
bash
```

```
# Clone and setup
git clone https://github.com/nexus-campus/nexus.git && cd nexus
npm install # installs all workspaces via Turborepo

# Start all infrastructure (PostgreSQL, Redis, Kafka, Elasticsearch)
docker-compose up -d

# Run migrations and seed SRM campus data
npm run db:migrate && npm run db:seed

# Start all services in parallel (Turborepo)
npm run dev

# Services available at:
# API Gateway: http://localhost:8000
# Admin Panel: http://localhost:3100
# Mobile:      Scan QR code in terminal with Expo Go app
```

14.4 Infrastructure Costs by Phase

Component	Year 1	Year 2	Year 3
Compute (ECS/EKS)	₹8,000	₹40,000	₹2,00,000
Database (RDS/Aurora)	₹5,000	₹25,000	₹1,00,000
Redis	₹2,000	₹10,000	₹40,000
Elasticsearch	₹3,000	₹15,000	₹60,000
CDN + S3	₹1,500	₹8,000	₹30,000
Kafka	₹2,000	₹12,000	₹50,000
Monitoring + Misc	₹3,500	₹10,000	₹20,000
Total/month	~₹25,000	~₹1,20,000	~₹5,00,000

15. SECURITY ARCHITECTURE

15.1 Defense-in-Depth

```
LAYER 1: Network
  Cloudflare WAF -> blocks OWASP Top 10 at edge
```

AWS Shield Standard → DDoS protection
VPC private subnets → databases never internet-exposed
Security Groups → least-privilege port access

LAYER 2: API

Rate limiting: 60 req/min per user; 1000/hr; 5 req/min for auth endpoints
Input validation: Zod schemas on all request bodies
SQL injection: Parameterized queries only (no string concat ever)
CORS: Strict allowlist (nexus.app, dev.nexus.app only)

LAYER 3: Application

Secrets: AWS Secrets Manager (never in code or env files in repos)
Dependencies: Snyk in CI/CD; Dependabot weekly PRs
Principle of Least Privilege: each service IAM role minimal
SAST: SonarCloud in every PR

LAYER 4: Data

Encryption at rest: AES-256 (RDS, S3, Redis)
Encryption in transit: TLS 1.3 everywhere
PII minimization: collect only what's necessary
Aadhaar: never stored – DigiLocker returns boolean only
Payment cards: never touch NEXUS servers (Razorpay handles entirely)

LAYER 5: Operational

Audit log: immutable append-only table, 7-year retention
Penetration testing: quarterly by third-party firm
Bug bounty: responsible disclosure program for students
Incident response: documented runbooks for all severity levels

15.2 OWASP Top 10 Mitigations

Threat	Mitigation
Broken Access Control	RBAC enforced at gateway AND service level; no client-side auth
Cryptographic Failures	TLS 1.3; bcrypt cost-12 for passwords; AES-256 at rest
Injection	Parameterized queries everywhere; ORM with query builder
Insecure Design	Threat modeling before each feature; security design review
Security Misconfiguration	Terraform enforces consistent config; CIS benchmark checks
Vulnerable Components	Snyk in CI; Dependabot auto-PRs
Auth Failures	15-min JWT; refresh rotation; 5-attempt logout + CAPTCHA

Threat	Mitigation
Data Integrity Failures	Signed JWTs; webhook HMAC verification; checksums
Logging Failures	Centralized Loki; immutable audit trail; SIEM integration
SSRF	No user-controlled URLs fetched server-side; strict allowlist

16. DEVELOPMENT PHASES & SPRINTS

Phase 1 — Foundation (Months 1-3)

Goal: Working, secure MVP that students can actually use.

Sprint 1-2 (Weeks 1-4):

- ☐ Monorepo + Docker Compose local environment
- ☐ CI/CD pipeline (GitHub Actions)
- ☐ Auth Service: register, login, OTP, JWT, sessions
- ☐ Student ID verification pipeline (AWS Textract)
- ☐ PostgreSQL schema (users, profiles, verifications)
- ☐ API Gateway (Kong) with rate limiting + auth validation

Sprint 3-4 (Weeks 5-8):

- ☐ Wallet Service (Go): balance, ledger, Razorpay integration
- ☐ Escrow: lock, release, refund, auto-release (BullMQ scheduled job)
- ☐ Webhook handler (idempotent, signature-verified)
- ☐ Wallet screens in mobile app

Sprint 5-6 (Weeks 9-12):

- ☐ Bazaar Service: listing CRUD, image upload (S3), search (Elasticsearch)
- ☐ Buy flow end-to-end (select listing → pay → escrow → confirm → release)
- ☐ Rating & review system
- ☐ Basic content moderation (Rekognition)
- ☐ Bazaar screens (list, detail, sell, manage)

☒ **Phase 1 Deliverable:** Students can register, verify identity, and buy/sell with escrow.

Phase 2 — Core Modules (Months 4-6)

Sprint 7-8:

- ☐ NEXUS FEAST: vendor onboarding portal, menu management, order flow

- ☐ Real-time order status (WebSocket: canteen tablet → buyer app)
- ☐ NEXUS Swift runner pool from Feast delivery runners

Sprint 9-10:

- ☐ NEXUS SWIFT: task board, task matching, GPS-verified completion
- ☐ NEXUS PULSE: event creation, ticket purchase, club profiles

Sprint 11-12:

- ☐ NEXUS SKILLS: service listing, portfolio, gig packages, milestone payment
- ☐ Dispute resolution workflow (Tier 1 auto + Tier 2 moderator portal)

☒ **Phase 2 Deliverable:** 5 modules live. Closed beta with 500 founding members.

Phase 3 — Rides & Advanced (Months 7-9)

Sprint 13-14:

- ☐ NEXUS RIDES (Go): ride board, matching algorithm, GPS tracking, SOS
- ☐ Driver verification: license, RC, insurance validation
- ☐ Women-only ride filter
- ☐ Pooled ride fare splitting

Sprint 15-16:

- ☐ Trust Score engine (nightly recompute + Redis sorted set)
- ☐ Fraud ML model v1 (deployed as FastAPI microservice)
- ☐ Campus moderator dashboard (web)
- ☐ Campus-custom map (OpenStreetMap + SRM POI layer)

Sprint 17-18:

- ☐ NEXUS Coins economy (earn, spend, leaderboard)
- ☐ Recommendation engine v1 (contextual, calendar-aware)
- ☐ Push notification system (Expo + FCM + APNs)

☒ **Phase 3 Deliverable:** Full super-app. Public launch at SRM. Marketing push begins.

Phase 4 — Optimize & Scale (Months 10-12)

Sprint 19-20:

- ☐ Performance: query optimization, Redis caching for hot paths
- ☐ App startup time < 2.5s; screen transitions < 200ms
- ☐ Load testing (k6): 10,000 concurrent users; fix bottlenecks

Sprint 21-22:

- ☐ Multi-campus architecture: schema-per-campus, shared global schema
- ☐ Campus onboarding portal (self-serve for new campuses)
- ☐ Analytics dashboards (Metabase internal + vendor dashboards)

Sprint 23-24:

- ☐ A/B testing framework (Firebase Remote Config)
- ☐ Feature flags for staged rollout
- ☐ DPDP Act compliance audit; penetration test by third-party
- ☐ Financial reconciliation tools + TDS automation
- ☒ **Phase 4 Deliverable:** Optimized, compliant platform ready for multi-campus expansion.

17. QUALITY ASSURANCE STRATEGY

17.1 Testing Pyramid



17.2 Critical Test Suites

```
typescript
```

```
// Wallet escrow – these must NEVER fail
describe('Escrow Flow', () => {
  test('locks exact amount from buyer', async () => {});
  test('prevents double-spend via concurrent requests', async () => {});
  test('releases to seller after confirmation', async () => {});
  test('refunds buyer on dispute resolution', async () => {});
  test('auto-releases after 24h if no dispute', async () => {});
  test('webhook processed exactly once (idempotency)', async () => {});
  test('rejects tampered payment amount from client', async () => {});
});

describe('Security', () => {
  test('student cannot access admin endpoints', async () => {});
  test('user cannot view another user wallet', async () => {});
  test('SQL injection attempt returns 400', async () => {});
  test('JWT with wrong signature is rejected', async () => {});
  test('rate limit triggers after threshold', async () => {});
});
```

17.3 Load Testing Targets

```
javascript

// k6 load test: simulate exam-week traffic spike
export const options = {
  stages: [
    { duration: '2m', target: 1000 }, // normal load
    { duration: '5m', target: 10000 }, // peak (lunch + exam week)
    { duration: '2m', target: 0 }, // cool down
  ],
  thresholds: {
    http_req_duration: ['p(99)<1000'], // 99% requests < 1 second
    http_req_failed: ['rate<0.01'], // < 1% error rate
  },
};
```

18. SCALING STRATEGY

18.1 Auto-Scaling Configuration

Service	Scale Trigger	Min	Max
Auth Service	CPU > 70%	2	8

Service	Scale Trigger	Min	Max
Bazaar Service	RPS > 300	2	12
Rides Service	Active rides > 300	2	10
Wallet Service	Error rate > 0.1%	3	6 (conservative)
WebSocket Service	Connections > 4,000/instance	2	20
Notification Service	Queue depth > 500	2	8

18.2 Database Scaling Path

Year 1: PostgreSQL primary + 1 read replica
→ Handles ~100K transactions/month comfortably

Year 2: PgBouncer connection pooling + 2 read replicas
→ Campus-schema separation; campus reads hit local replica

Year 3: Aurora PostgreSQL Serverless v2 (auto-scaling storage + compute)
→ Aurora Global DB for cross-region (User + Wallet tables)
→ Regional clusters for campus-specific data

Year 5+: Full horizontal sharding by campus_id
→ Vitess or CitusDB for distributed PostgreSQL
→ Global DB for User, Wallet, Trust (cross-campus entities)

18.3 CDN & Image Optimization

All listing images:

1. Upload: raw image → S3 (private bucket)
2. Lambda trigger: resize to 400x400, 800x800, 1200x1200 WebP variants
3. Serve: CloudFront CDN with cache-control: 7 days
4. App: progressive loading (blurhash placeholder → small → full)
5. Result: 60-80% bandwidth reduction vs serving raw images

19. DISASTER RECOVERY

19.1 Recovery Objectives

System	RTO	RPO
Wallet + Transactions	< 5 min	0 sec (synchronous replication)

System	RTO	RPO
Auth + User data	< 10 min	< 30 sec
Listings + Marketplace	< 30 min	< 5 min
Analytics	< 4 hours	< 24 hours

19.2 Backup Strategy

- **PostgreSQL:** Continuous WAL archiving to S3 + daily automated snapshots (30-day retention)
- **Redis:** RDB snapshots every 15 minutes + AOF persistence
- **S3:** Cross-region replication enabled for all buckets
- **Elasticsearch:** Daily snapshots to S3

19.3 Incident Severity & Response

```
SEV-1: Payment outage
  Detection: PagerDuty alert (payment success < 95% for 2 min)
  Response: On-call engineer paged in < 2 min
  Action: Switch to backup payment gateway via feature flag
  Comms: In-app banner "Payment processing delayed"
  Post-mortem: Within 48 hours

SEV-2: Service degradation (error rate > 5%)
  Detection: Grafana alert
  Response: Auto-scale first; engineer on-call if no improvement in 5 min
  Action: Feature flag to disable affected module gracefully
  User impact: "Temporarily unavailable" screen for affected module

SEV-3: Performance degradation (p99 > 2s)
  Detection: Grafana alert
  Response: Engineer investigates in business hours
  Action: Database query optimization, cache warm-up
```

20. COMPLIANCE ENGINEERING

20.1 DPDP Act 2023 Implementation

```
typescript
```

```

// Right to Access – export all user data
app.get('/api/v1/user/data-export', authenticate, async (req, res) => {
  const userId = req.user.id;
  const [profile, listings, transactions, ratings] = await Promise.all([
    userService.getUserProfile(userId),
    bazaarService.getUserListings(userId),
    transactionService.getUserHistory(userId),
    ratingService.getUserRatings(userId),
  ]);

  res.json({
    generated_at: new Date().toISOString(),
    data: { profile, listings, transactions, ratings }
  });
});

// Right to Erasure
app.post('/api/v1/user/delete-account', authenticate, async (req, res) => {
  const userId = req.user.id;
  // Anonymize (not delete) – financial records required for 7 years
  await userService.anonymize(userId);
  // Anonymization: name → "Deleted User", email → hash, phone → hash
  // All other personal data deleted; financial records retained with anonymous ref
  await authService.revokeAllSessions(userId);
  res.json({ status: 'deletion_scheduled', completes_at: addDays(new Date(), 30) });
});

```

20.2 Audit Trail (Immutable)

sql

```

-- Append-only audit log
CREATE TABLE audit_log (
  id BIGSERIAL PRIMARY KEY,
  timestamp TIMESTAMPTZ DEFAULT NOW() NOT NULL,
  user_id UUID,
  action VARCHAR(100) NOT NULL,
  resource_type VARCHAR(50),
  resource_id UUID,
  ip_address INET,
  request_id UUID,
  metadata JSONB DEFAULT '{} '
) PARTITION BY RANGE (timestamp);

-- DB-level constraint: no UPDATE or DELETE for app role
REVOKE UPDATE, DELETE ON audit_log FROM nexus_app;
-- Retained 7 years; replicated to read-only audit DB

```

20.3 FSSAI Vendor Validation

```

typescript

// Auto-validate vendor FSSAI license on onboarding and quarterly renewal
async function validateFSSAI(licenseNo: string): Promise<FSSAIResult> {
  const res = await fetch(
    `https://foscoss.fssai.gov.in/api/licenseInfo?licenseNo=${licenseNo}`
  );
  const data = await res.json();
  return {
    isValid: data.status === 'Active',
    expiryDate: data.expiryDate,
    businessName: data.businessName,
  };
}

// Cron: check all vendor licenses weekly
// Expired: suspend vendor, notify via email + push, 7-day grace period

```

21. ADMIN & OPERATIONS TOOLS

21.1 Campus Admin Portal

Tech: Next.js + shadcn/ui + Recharts

Modules:

Module	Features
Dashboard	Real-time GMV, DAU, transaction success rate, active disputes
User Management	Search, view full profile, verify manually, suspend, ban
Listing Moderation	Flagged content queue, approve/reject, bulk actions
Dispute Center	All open disputes, evidence viewer (secure), resolution tools
Vendor Management	Approve applications, view analytics, manage FSSAI status
Financial Overview	Revenue by module, escrow float, payout history
Announcements	Push campus-wide notifications, create notices
Analytics	Custom date range, cohort analysis, module breakdown

21.2 Vendor Dashboard

- Live order queue (accept → preparing → ready)
- Menu management (real-time availability toggle)
- Revenue analytics (daily, weekly, monthly)
- Customer ratings + review management
- Bank payout management + history
- FSSAI license renewal reminders

21.3 Moderator Tools

- Priority-sorted flagged content queue
- Context panel: user history, past violations, trust score
- One-click approve/reject with reason tagging
- Bulk spam campaign detection and action
- Escalation to campus admin with full case file

22. THIRD-PARTY INTEGRATIONS

Integration	Provider	Purpose
Payment Gateway (Primary)	Razorpay	UPI, cards, net banking, payouts

Integration	Provider	Purpose
Payment Gateway (Backup)	Cashfree	Failover if Razorpay has outage
SMS OTP	MSG91 / Twilio	Phone number verification
Email	AWS SES	Transactional emails
Push Notifications	Expo + FCM + APNs	iOS + Android push
Campus Maps	Google Maps SDK + Custom OSM	Navigation + campus POIs
Image Moderation	AWS Rekognition	NSFW + prohibited item detection
OCR (ID Verification)	AWS Textract	Student ID card text extraction
Face Match	AWS Rekognition	Selfie ↔ ID card photo verification
Aadhaar Verification	DigiLocker API (NIC)	Government identity verification
Vendor Payouts	Razorpay X	Bulk payouts to vendor bank accounts
Product Analytics	Mixpanel	Funnels, retention, A/B test results
Error Tracking	Sentry	Real-time crash + error monitoring
APM	Datadog	Distributed tracing, performance monitoring
CDN + WAF	Cloudflare	Edge caching + DDoS protection
FSSAI Validation	FOSCOS API	Food vendor license verification

23. POST-LAUNCH MAINTENANCE

23.1 Release Cadence

Type	Frequency	Contents
Hotfix	As needed (< 4h)	Critical bugs, security patches
Patch	Weekly	Minor fixes, performance tweaks
Minor Release	Bi-weekly	Feature improvements, UX updates
Major Release	Monthly	New modules, architectural changes

23.2 Feature Flag Rollout

New Feature → Feature Flag OFF (all users)
→ Internal team testing (flag ON for team)
→ Campus Crew beta (1% rollout)
→ 5% → 20% → 50% → 100% gradual rollout
→ A/B testing variant at 50/50 split during rollout
→ Monitor: conversion rate, error rate, support tickets
→ Kill switch: instant rollback if metrics drop

23.3 Technical Debt Policy

- 20% of every sprint dedicated to refactoring
- Quarterly "tech debt sprint" (entire sprint, no new features)
- SonarCloud quality gate: PRs blocked if code coverage drops below 70%
- Dependabot auto-PRs: reviewed and merged weekly

23.4 Feedback Loop

User Feedback Sources:

1. In-app 5-star after each transaction (instant signal)
2. NPS survey – quarterly via push notification
3. In-app bug report button (all screens)
4. Campus crew qualitative interviews (monthly)
5. Sentry crash reports (automatic)
6. PostHog session recordings (anonymized heatmaps)

Cadence:

Weekly → Product team triage all feedback
Monthly → Roadmap update based on feedback patterns
Quarterly → Major product review + direction setting

APPENDIX: Port Map (Local Development)

Service	Port
API Gateway	8000
Auth Service	3001
Bazaar Service	3002

Service	Port
Wallet Service (Go)	3003
Feast Service	3004
Rides Service (Go)	3005
Swift Service	3006
Skills Service	3007
Pulse Service	3008
Trust Service	3009
Notification Service	3010
Search Service	3011
Analytics Service (Python)	3012
Admin Dashboard	3100
PostgreSQL	5432
Redis	6379
Kafka	9092
Elasticsearch	9200